Software Security

University of Insubria

Software vulnerabilities

A vulnerability is a weakness in a program that can be exploited by an attacker to perform unauthorized actions

- To exploit a vulnerability, attackers use tools and techniques related to the weakness in the software

Software vulnerabilities

```
int autenticate() {
    char* password = "MyPassword!";
    char* input = malloc(256);

    printf("Enter the password: ");
    scanf("%s",input);
    if (strcmp(password,input)==0) {
        printf("Authenticated!\n");
        return 1;
    } else {
        printf("The password is wrong!\nPlease, try again!\n");
        return 0;
    }
}
```

Software vulnerabilities

There are two vulnerabilities in this code

- Hardcoded password
- Potential buffer overflow

```
int autenticate() {
    char* password = "MyPassword!";
    char* input = malloc(256);

    printf("Enter the password: ");
    scanf("%s",input);
    if (strcmp(password,input)==0) {
        printf("Authenticated!\n");
        return 1;
    } else {
        printf("The password is wrong!\nPlease, try again!\n");
        return 0;
    }
}
```

Bug, Error, Fault, and Failure

Bug

- An error or a fault that causes a failure in the program

Error

- A mistake made by a human that leads to incorrect results

Fault

- An incorrect step, process, or data definition in a computer program

Failure

- When the software can't perform as expected or meet performance requirements

Code vulnerabilities

Information leakage

- Unintentional disclosure of sensitive data, which attackers can use to exploit security vulnerabilities

```
int autenticate() {
    char* password = "MyPassword!";
    char* input = malloc(256);

    printf("Enter the password: ");
    scanf("%s",input);
    if (strcmp(password,input)==0) {
        printf("Authenticated!\n");
        return 1;
    } else {
        printf("The password is wrong!\nPlease, try again!\n");
        return 0;
    }
}
```

Buffer overflow

- When data exceeds a buffer's boundary and overwrites adjacent memory locations
  This may cause:
  - execution of malicious code
  - privileges escalation (gaining higher access rights)

Code vulnerabilities

```c
int raceCondition() {
  char * fn = "/tmp/XYZ";
  char buffer[60];
  FILE *fp;
  if(!access(fn, W_OK)){
    scanf("%50s", buffer );
    fp = fopen(fn, "a+");
    fwrite("\n", sizeof(char), 1, fp);
    fwrite(buffer, sizeof(char), strlen(buffer), fp);
    fclose(fp);
  } else {
    printf("No permission \n");
  }
}
```

Code vulnerabilities

Race condition

- The small time gap between security control application and use of services in a system, allowing attackers to change program behavior
  - caused by interference between multiple threads sharing the same resources

Invalid data processing

- When data is processed with incorrect or incomplete assumptions, enabling attackers to trigger unwanted behaviors or execute malicious code

```
int raceCondition() {
  char * fn = "/tmp/XYZ";
  char buffer[60];
  FILE *fp;
  if(!access(fn, W_OK)){
    scanf("%50s", buffer );
    fp = fopen(fn, "a+");
    fwrite("\n", sizeof(char), 1, fp);
    fwrite(buffer, sizeof(char), strlen(buffer), fp);
    fclose(fp);
  } else {
    printf("No permission \n");
  }
}
```

How to avoid vulnerabilities?

Secure programming

- ensures software is protected from vulnerabilities, attacks, and anything that could harm the system or software

Techniques

- **Defensive programming**
  - Involves checking for errors or attacks early (e.g., identifying preconditions to prevent issues)
- **Security by Design**
  - Security should be a priority at every stage of software development (e.g., threat modeling)

Static and Dynamic Analysis

Static Program Analysis

Static analysis examines software without executing it

- Can be done on source code or object code

Tools

- strings
  - Extracts human-readable strings from a binary
- objdump
  - Displays information about object files
- readelf
  - Reads and displays information from ELF files

Using Binary Information to Discover Vulnerabilities

Analyze binaries to check for vulnerabilities

- Input validation
  - ensure inputs are properly validated
- Memory management
  - verify if memory is safely managed
- Up-to-date libraries
  - check if the latest libraries or frameworks are used
- Compilation details
  - review how the application is compiled
- Sensitive data handling
  - check if (static) strings are used to store sensitive information

Static Program Analysis

Program: guessmyname

Functionality: When executed, the program asks the user to guess a name

```
CC> ./guessmyname
Guess my name: Paulo
I am sorry, this is not my name!
Please, try again.
CC>
```

Static Program Analysis

.rodata: read-only data (e.g., constants, strings)

It is possible to use **readelf** to check if the name to guess is hardcoded in the file

- We can read the content of .rodata section

```
CC> readelf -x .rodata guessmyname

Hex dump of section '.rodata':
  0x00002000 01000200 00000000 43616c65 66004775 ........Calef.Gu
  0x00002010 65737320 6d79206e 616d653a 20002531 ess my name: .%1
  0x00002020 30730000 00000000 57656c6c 20646f6e 0s......Well don
  0x00002030 65212059 6f752068 61766520 67756573 e! You have gues
  0x00002040 73656420 6d79206e 616d6521 00000000 sed my name!....
  0x00002050 4920616d 20736f72 72792c20 74686973 I am sorry, this
  0x00002060 20697320 6e6f7420 6d79206e 616d6521  is not my name!
  0x00002070 00506c65 6173652c 20747279 20616761 .Please, try aga
  0x00002080 696e2e00                             in..
```

Static Program Analysis

Program: guessmyname

Functionality: When executed, the program asks the user to guess a name

Vulnerabilities: Information leakage

```
CC> ./guessmyname
Guess my name: Calef
Well done! You have guessed my name!
CC>
```

Static Program Analysis

It is possible to use **strings** to extract all the constant strings used in the code

```
CC> strings guessmyname
/lib64/ld-linux-x86-64.so.2
libc.so.6
__isoc99_scanf
puts
printf
malloc
__cxa_finalize
strcmp
__libc_start_main
GLIBC_2.7
GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
u+UH
[]A\A]A^A_
Calef
Guess my name:
%10s
Well done! You have guessed my name!
I am sorry, this is not my name!
Please, try again.
```

Static Analysis

```
int main(int argc, char** argv) {
        char* name = "Calef";
        char* input = malloc(10);

        printf("Guess my name: ");
        scanf("%10s",input);
        if (strcmp(name,input)==0) {
                printf("Well done! You have guessed my name!\n");
        } else {
                printf("I am sorry, this is not my name!\n");
                printf("Please, try again.\n");
        }
}
```

Dynamic Program Analysis

Dynamic analysis involves observing a program's behavior while it's running on a processor (virtual or real)

Types of Dynamic Analysis

- **Dynamic Memory Analysis**
  - Monitors memory usage during execution
- **Dynamic Disassembly**
  - Analyzes the program's instructions as they execute

Dynamic Program Analysis

Program: guessmyname2

Functionality: When executed, the program asks the user to guess a name

Vulnerabilities: ~~Information leakage~~

```
CC> ./guessmyname2
Guess my name: Calef
I am sorry, this is not my name!
Please, try again.
CC>
```

Dynamic Program Analysis

.rodata: read-only data (e.g., constants, strings)

It is possible to use **readelf** to check if the name to guess is hardcoded in the file

- We can read the content of .rodata section

```
CC> readelf -x .rodata guessmyname2

Hex dump of section '.rodata':
  0x00002000 01000200 00000000 47756573 73206d79 ........Guess my
  0x00002010 206e616d 653a2000 25313073 00000000  name: .%10s....
  0x00002020 57656c6c 20646f6e 65212059 6f752068 Well done! You h
  0x00002030 61766520 67756573 73656420 6d79206e ave guessed my n
  0x00002040 616d6521 00000000 4920616d 20736f72 ame!....I am sor
  0x00002050 72792c20 74686973 20697320 6e6f7420 ry, this is not
  0x00002060 6d79206e 616d6521 00506c65 6173652c my name!.Please,
  0x00002070 20747279 20616761 696e2e00           try again..
```

Dynamic Program Analysis

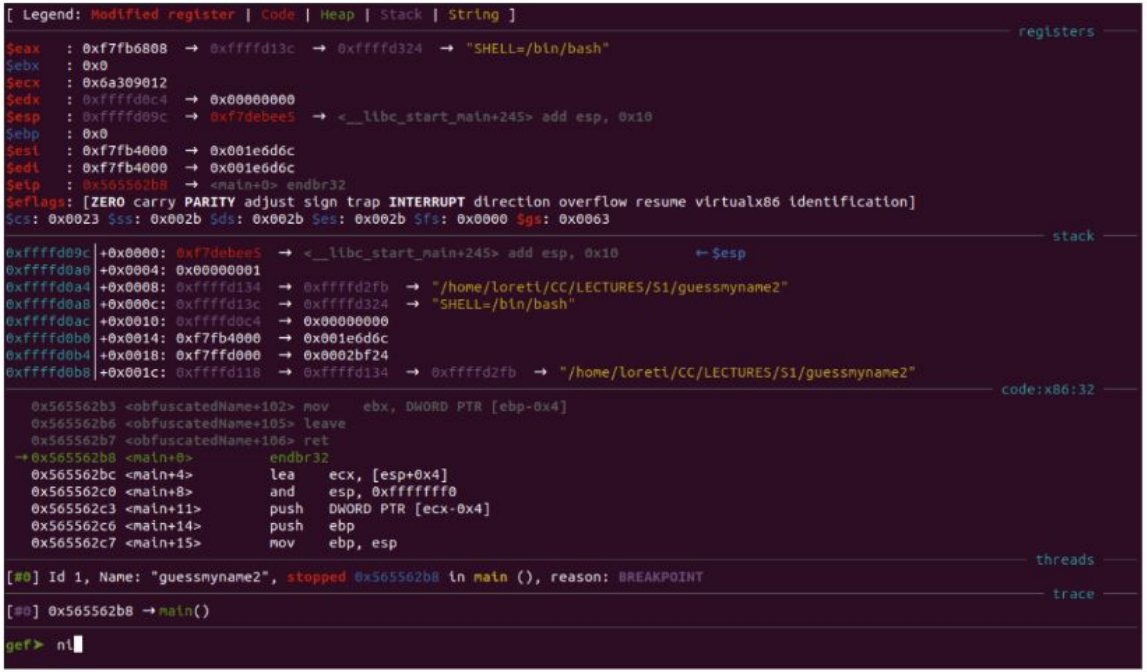
Dynamic program analysis with GDB/GEF

- start gdb, load the binary, and set a breakpoint at function main

```
GEF for linux ready, type `gef' to start, `gef config' to configure
92 commands loaded for GDB 9.2 using Python engine 3.8
gef➤  file guessmyname2
Reading symbols from guessmyname2...
(No debugging symbols found in guessmyname2)
gef➤  break main
Breakpoint 1 at 0x12b8
gef➤  run
```

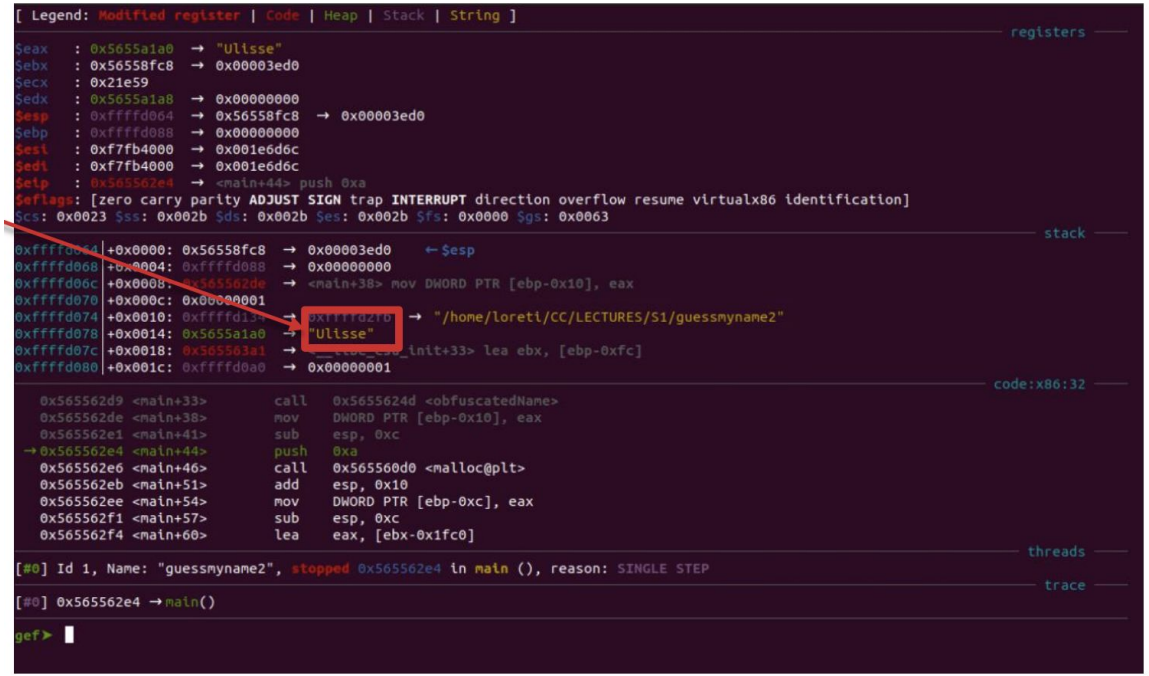
Dynamic Program Analysis

Executing the program step-by-step (command nexti - ni)



Dynamic Program Analysis

Executing the program step-by-step (command nexti - ni)



Dynamic Program Analysis

Program: guessmyname2

Functionality: When executed, the program asks the user to guess a name

Vulnerabilities: Information leakage

```
CC> ./guessmyname2
Guess my name: Ulisse
Well done! You have guessed my name!
CC>
```

Dynamic Program Analysis

Program: guessmyname2

Functionality: When executed, the program asks the user to guess a name

```
int main(int argc, char** argv) {
        char* name = obfuscatedName();
        char* input = malloc(10);

        printf("Guess my name: ");
        scanf("%10s",input);
        if (strcmp(name,input)==0) {
                printf("Well done! You have guessed my name!\n");
        } else {
                printf("I am sorry, this is not my name!\n");
                printf("Please, try again.\n");
        }
}
```

Dynamic Program Analysis

Program: guessmyname2

Functionality: When executed, the program asks the user to guess a name

```
int main(int argc, char** argv) {
        char* name = obfuscatedName();
        char* input = malloc(10);

        printf("Guess my name: ");
        scanf("%10s",input);
        if (strcmp(name,input)==0) {
                printf("Well done! You have guessed my name!\n");
        } else {
                printf("I am sorry, this is not my name!\n");
                printf("Please, try again.\n");
        }
}
```

It is possible to patch the program?

Dynamic Program Analysis

Program: guessmyname3

Functionality: When executed, the program asks the user to guess a name

```
int main() {
    char* name = readEncryptedName();
    char* input = malloc(10);

    printf("Guess my name: ");
    scanf("%10s",input);
    if (strcmp(name,encrypt(input))==0) {
        printf("Well done! You guessed my name!\n");
    } else {
        printf("I am sorry, this is not my name!\nPlease, try again!\n");
    }
    return 0;

}
```

Reverse Engineering

Reverse Engineering

Normal Process

- Compilers, assemblers, and linkers are used to create executable programs

Reverse process

- Given an executable, we want to obtain the assembly or even the source code that created it

Tools

- Disassemblers: Convert executables to assembly code
- Decompilers: Attempt to convert executables back into high-level source code

Disassembler for Vulnerability Analysis

Disassembly tools are commonly used for

- Malware analysis
- Analyzing closed-source software to **identify vulnerabilities**
- Display program instructions while debugging

The security auditing process consists of three steps

- Vulnerability Discovery
- Vulnerability Analysis
- Exploit Development

Disassembler for Vulnerability Analysis

After discovering a vulnerability

- Is it exploitable?
- How can it be exploited?

A disassembler can be used to gather crucial information, such as

- Allocated memory
- Order of allocated variables

Vulnerability exploitation if often based on the combination of debugger with a disassembler

Disassembly

Disassembler for Vulnerability Analysis

Program: findmysecret

Functionality: None

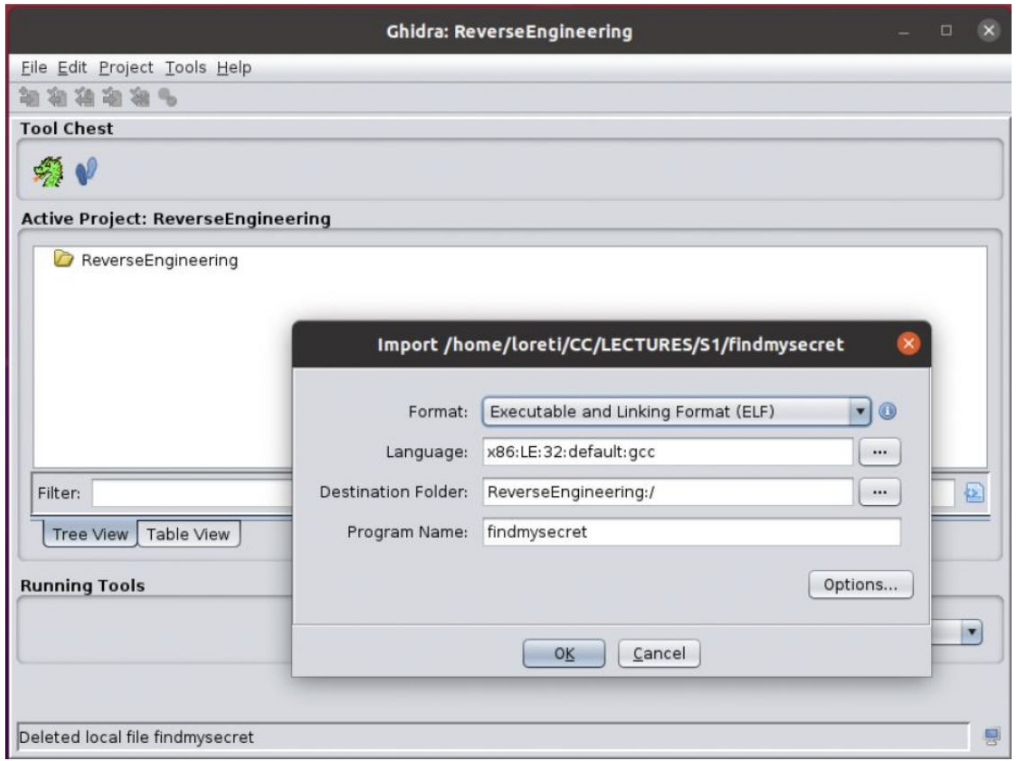
```
CC> ./findmysecret
You will never know my secret!
CC>
```

Disassembly

Ghidra for Information Recovery

Steps

- Create a new project
- Import the program (binary file) into the project
- Analyze the code to collect information from the binary



Disassembly

Ghidra for Information Recovery

Accessing function list

- view the list of functions defined in the program

Find specific functions

- among the functions, you can locate functionOne

```
*************************************************************
*                          FUNCTION                         *
*************************************************************
undefined functionOne()
    undefined         AL:1           <RETURN>
    undefined4        Stack[-0x8]:4  local_8                 XREF[1
                      functionOne                    XREF[3]:     En
                                                                  00
000111ef f3 0f 1e fb     ENDBR32
000111f3 55              PUSH        EBP
000111f4 89 e5           MOV         EBP,ESP
000111f6 53              PUSH        EBX
000111f7 83 ec 04        SUB         ESP,0x4
000111fa e8 d1 fe        CALL        __x86.get_pc_thunk.bx
         ff ff
000111ff 81 c3 d9        ADD         EBX,0x2dd9
         2d 00 00
00011205 e8 c3 ff        CALL        get
         ff ff
0001120a 83 ec 0c        SUB         ESP,0xc
0001120d 50              PUSH        EAX
0001120e e8 5d fe        CALL        FUN_00011070
         ff ff
00011213 83 c4 10        ADD         ESP,0x10
00011216 90              NOP
00011217 8b 5d fc        MOV         EBX,dword ptr [EBP + local_8]
0001121a c9              LEAVE
0001121b c3              RET
```

Disassembly

GDB/GEF to Call *functionOne*

Steps

- Load the program in GDB/GEF
- Set a breakpoint in the main function (break main)
- Run the program (run), and when it hits the breakpoint, use the call command to call functionOne

```
gef➤  file findmysecret
Reading symbols from findmysecret...
(No debugging symbols found in findmysecret)
gef➤  break main
Breakpoint 1 at 0x1234
gef➤  run
```

```
[#0] 0x56556234 → main()

gef➤  call (void *) functionOne()
My secret is 42!

$2 = (void *) 0x12
gef➤
```

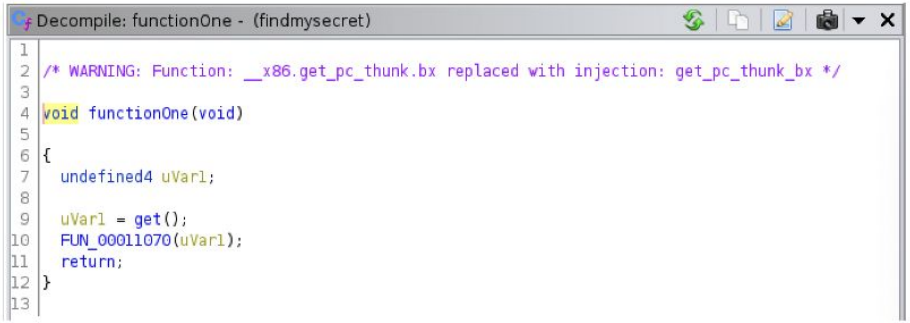
Decompilers and Ghidra

A decompiler is a tool that attempts to reconstruct high-level source code from a binary file

- Easier for languages like Java
- Harder for general binary files, especially with obfuscation

Ghidra includes an integrated decompiler (requires some interactions)

- Functions signature (return type and arguments)
- Data types
- Variable names and types



Challenges

SS_0.01 - The safe
SS_0.02 - acrostic
SS_0.03 - dissection
SS_0.04 - volatility
SS_0.05 - piecewise

Challenges

SS_1.01 - NextGen Safe
SS_1.02 - Slow Printer
SS_1.03 - Flag Checker
SS_1.04 - Unbreakable AES
SS_1.05 - morph
SS_1.06 - pacman